

AegisCare HMS / LabTrack v5.5

System	AegisCare HMS / LabTrack v5.5
Applies to	FACILITY_NAME (FACILITY_ADDRESS), all users of the AegisCare deployment, and all processing of data within the Supabase project tgynjasgnerucrlwedu
Policy owner	Facility In-Charge / Information Security Officer (to be named)
Document status	DRAFT for DHA certification submission
Date	2026-08-12
Related documents	DOC-2 (DPIA), DOC-3 (Privacy Policy), DOC-4 (Backup & Recovery), DOC-5 (FHIR Integration), DOC-7 (Gap Analysis)

Section 1 — Authentication & Session Management

1.1 Authentication provider. All access uses Supabase Auth (email/password) with JWT sessions persisted client-side (src/integrations/supabase/client.ts: persistSession: true, autoRefreshToken: true). Passwords are stored by Supabase Auth as salted, hashed credentials (bcrypt-class hashing, platform default); the application never stores or transmits plain-text passwords.

1.2 Account provisioning. User accounts are created only by administrators through server functions (src/lib/admin-users.functions.ts) using the service-role admin API (supabaseAdmin.auth.admin.createUser), protected by the requireSupabaseAuth middleware and an assertAdmin server-side check. The service-role key is never exposed to the browser. Self-signup is disabled; a pending access_requests queue exists for administrator review.

1.3 Session timeout. Automatic logout after 30 minutes of inactivity (src/components/app-shell.tsx, TIMEOUT_MS = 30 * 60 * 1000), monitored on mouse, keyboard, touch, and scroll events. Satisfies the DHA requirement for automatic session termination.

1.4 Password policy. Enforced client-side (Zod) on password creation and change (src/routes/change-password.tsx):

- minimum 8 characters; at least one uppercase letter; at least one number; at least one special character; maximum 72 characters.

1.5 Deferred items (disclosed):

- ☐ PENDING: server-side password policy (Supabase dashboard setting) — requires transfer to a Supabase plan with dashboard access (documented in docs/dha-compliance-task-d-phase2-security.md).
- ☐ PENDING: MFA/TOTP enforcement — available when the project moves to a Supabase plan supporting Auth MFA; frontend TOTP UI is planned.

Section 2 — Role-Based Access Control

2.1 Role model. Nineteen roles defined in src/lib/roles.ts and the public.app_role enum: admin, system_admin, staff, receptionist, accountant, insurance_agent, records_officer, triage_nurse, nurse, doctor, clinical_officer, dental_officer, nutritionist, physiotherapist, hts_counsellor, lab_tech, radiologist, pharmacist, mortician (20260730140000_dha_task_d_security_roles.sql).

2.2 Enforcement layers.

- Database (primary): `is_approved(auth.uid())` — SECURITY DEFINER, pinned `search_path = public` — gates every RLS policy (≈ 134 policy statements across the schema). The 2026-07-30 role-gap fix (20260730140000...) expanded `is_approved()` to all 19 roles, repairing 68 policies that previously denied clinical roles.
- Feature layer: `user_has_permission(_user, _perm)` over `role_permissions` (e.g., `sign_encounter` granted to `doctor`, `clinical_officer`, `dental_officer`, `admin` — Sprint 13A).
- UI layer: Guard / PermGuard / `useHasPerm` (`src/lib/require-access.tsx`, `src/lib/auth-context.tsx`).
- Room level: `user_room_access` + `can_access_room()` — staff can only open rooms they are granted.

2.3 Privilege separation. Admin-only actions (user create/update/delete, role assignment, room management, pricing) require the admin role AND server-side assertion (`assertAdmin` in `src/lib/admin-users.functions.ts`). Audit log and role tables themselves are audited (`user_roles`, `role_permissions` covered by `audit_trigger_fn` — Task E).

Section 3 — Row-Level Security

3.1 RLS is enabled on every clinical, financial, and security-relevant table. Representative policies (all in `supabase/migrations/`):

Table(s)	Policy pattern
encounters, patients, invoices, prescriptions, admissions, clinical_notes, lab_tests (legacy), stock_*, deliveries, machines, machine_logs, fund_utilizations, rooms, role_permissions	Approved ... / Authenticated ... CRUD policies keyed on <code>is_approved(auth.uid())</code>
lab_orders, lab_results	<code>lab_orders_*</code> / <code>lab_results_*</code> — <code>SELECT/INSERT/UPDATE/DELETE</code> for approved users (20260806000005_rls_missing_tables.sql)
encounter_room_visits	<code>encounter_room_visits_*</code> CRUD for approved users
icd11_codes, moh_indicator_definitions, room_indicator_map	Read-only reference policies
profiles	<code>profiles_select_authenticated</code> (<code>SELECT</code> for authenticated), <code>profiles_insert_own</code> / <code>profiles_update_own</code> (self only)
audit_log	<code>audit_log_insert_authenticated</code> (<code>INSERT</code>), <code>UPDATE/DELETE</code> denied (<code>audit_log_deny_update</code> , <code>audit_log_deny_delete</code>) — append-only
audit_log_archive, audit_archive_runs	<code>audit_archive_select</code> (approved <code>SELECT</code>); <code>INSERT/UPDATE/DELETE</code> denied
shr_transmission_log	<code>shr_log_select</code> ; <code>INSERT/UPDATE/DELETE</code> denied — append-only
sha_claims, sha_claim_items, sha_claim_packages	<code>sha_claims_authenticated</code> etc. — FOR ALL with <code>is_approved(auth.uid())</code>
sha_benefit_packages	<code>sha_benefit_packages_authenticated_read</code> (<code>SELECT</code> authenticated), <code>sha_benefit_packages_admin_write</code>
consent_otps	approved users can read/insert/update otps

Table(s)	Policy pattern
dha_outbound_queue	dha_outbound_queue_authenticated — FOR ALL approved
moh_705_disease_mappings	moh_705_disease_mappings_authenticated (SELECT approved; public read removed in hardening)
mortuary_records	approved_read_mortuary, approved_write_mortuary
medication_administrations	approved_read/insert/update_med_admin
access_requests, user_room_access	Self-scoped (Users can view their own access request, Users view their own room access)

3.2 Hardening migration 20260811000007_security_hardening.sql:

- removed permissive public/anon INSERT and SELECT policies on audit_log;
- restricted profiles SELECT to authenticated;
- restricted moh_705_disease_mappings to authenticated approved;
- rebuilt stock_store_usage_view without joining auth.users;
- revoked anon EXECUTE on eight SECURITY DEFINER functions (generate_fhir_encounter, get_moh_705_report, validate_and_get_icd11, verify_patient_identity, verify_practitioner, get_contracted_price, create_encounter_from_appointment, enforce_encounter_lock);
- pinned search_path = public on twelve SECURITY DEFINER functions;
- added a verification DO block that fails the migration if permissive audit_log INSERT policies reappear.

Section 4 — Audit Logging

4.1 audit_log records table_name, record_id, action (INSERT/UPDATE/DELETE/BREAK_GLASS), full old_data/new_data JSONB snapshots, changed_by (Supabase auth.uid()), changed_at (20260730150000_dha_task_e_audit_completeness.sql).

4.2 Coverage: 20 tables / 60 triggers via audit_trigger_fn(): clinical (admissions, encounters, lab_tests, lab_orders, lab_results, radiology_orders, radiology_results, clinical_notes, encounter_diagnoses, prescriptions, patients), financial (invoices, invoice_line_items, invoice_payments), security (user_roles, role_permissions, user_room_access, app_settings), pharmacy/stock (stock_items, stock_movements). Excluded intentionally: computed aggregates, reference data, and the audit log itself.

4.3 Immutability: RLS denies UPDATE and DELETE to all roles; writes occur via SECURITY DEFINER triggers. Archiving is the only removal path (see 4.5).

4.4 Break-glass: log_break_glass_access(patient_id, justification, user, email) writes a BREAK_GLASS audit event with mandatory justification and queues a client notification (20260806000001_sprint8a_break_glass.sql, 20260806000002_sprint8c_shr_access_notification.sql).

4.5 Archiving & retention: archive_old_audit_logs() (SECURITY DEFINER) moves rows older than 2 years to audit_log_archive (append-only), recording each run in audit_archive_runs; scheduled nightly by cron job archive-audit-logs-nightly (0 23 * * * UTC = 02:00 EAT) (20260806101050_e8638a1f-75b6-42c3-9a64-be6489877920.sql).

Archived rows are retained indefinitely; the 20-year retention statement is formalised in DOC-3 §6. [△] GAP: a written retention/legal-hold procedure is still to be signed off (administrative).

4.6 SHR transmission metadata log: shr_transmission_log — append-only, written by trg_shr_transmission_log / trg_shr_transmission_log_insert on dha_outbound_queue transitions for fhir_sync and shr_access_notification types (Digital Health (Data Exchange Component) Regulations 2025).

4.7 Review: src/routes/admin.audit-log.tsx provides administrator review UI.

Section 5 — Data Encryption

Requirement Implementation	
Encryption in transit	TLS 1.2+ enforced by Supabase and Lovable hosting (HTTPS-only)
Encryption at rest	AES-256 — Supabase platform default for PostgreSQL and Storage
Secrets	Edge-function secrets stored in Supabase only (ICD_CLIENT_ID, ICD_CLIENT_SECRET, etc.); never in the repository (.env contains only the publishable key)
OTP storage	consent_otps.otp_hash stores SHA-256 digests only — plaintext OTPs are never persisted

Section 6 — Vulnerability Management

6.1 Dependency scanning & patching. All CVEs identified by the Lovable security scan are remediated in package.json:

- ❑ seroval CVE-2026-23737 — fixed via overrides: { "seroval": "^1.5.4" } (+ patches/ via patch-package).
- ❑ xlsx CVE-2023-30533 and CVE-2024-22363 — dependency removed (Excel export replaced).
- ❑ No other known-vulnerable runtime dependencies at the time of writing.

6.2 CI guard: .github/workflows/ci.yml runs lint, tests, and production build on every push/PR with bun install --frozen-lockfile, preventing lockfile drift and unvetted dependency introduction.

6.3 Review cadence: dependency audit quarterly; on any new advisory affecting the runtime stack, fix within 30 days (90 days for low-severity).

Section 7 — Edge Function Security

7.1 JWT verification: all five edge functions (claims-dispatcher, fhir-patient, fhir-encounter, fhir-condition, icd11-search) are deployed with verify_jwt = true (supabase/config.toml); requests without a valid Supabase JWT are rejected at the platform layer.

7.2 Caller authentication: claims-dispatcher additionally validates the caller via supabase.auth.getUser() with the presented Authorization header before any queue write.

7.3 Least privilege & secrecy: edge functions use the service-role client server-side only; external secrets (WHO ICD-11 client credentials) are held in Supabase secrets and never returned to clients. icd11-search caches the WHO token in memory and refreshes before expiry.

7.4 CORS: permissive Access-Control-Allow-Origin: * headers are used at the function layer to permit the Lovable origin; combined with JWT verification this exposes no data without a valid token. ⚠ GAP (hardening recommendation): restrict CORS to the known application origins (<https://aegiscarehms.lovable.app> and the facility deployment URL) before production certification.

| Section 8 — Database Function Security

8.1 SECURITY DEFINER discipline. Trigger/RPC functions that must write across RLS boundaries run SECURITY DEFINER (e.g., `audit_trigger_fn()`, `generate_fhir_encounter()`, `auto_generate_sha_claim()`, `sync_encounter_diagnoses_from_jsonb()`, `trg_medication_dispense_fhir()`, `log_break_glass_access()`, `archive_old_audit_logs()`). For every such function the `search_path` is pinned to `public` (twelve functions explicitly fixed in `20260811000007_security_hardening.sql`; others declare `SET search_path` at creation) to prevent search-path hijacking.

8.2 Executable surface. `anon EXECUTE` was revoked on all eight data-returning SECURITY DEFINER functions (Section 3.2); RLS-gate functions (`is_approved`, `has_role`, `user_has_permission`) are executable only by authenticated (`20260608060332...`, `20260609052310...`).

8.3 Accepted platform limitation. A security-scan warning exists for `anon EXECUTE` on SECURITY DEFINER functions that the Lovable platform itself requires (for its managed flows). This is an accepted platform constraint, documented and reviewed quarterly — not a code defect. The functions in question return no patient-identifiable data under `anon` and are revoked where functionally possible.

| Section 9 — Incident Response

9.1 Incident classes: unauthorised access, data breach/exfiltration, audit log tampering attempts, break-glass misuse, malware/phishing on staff workstations, credential compromise, service outage.

9.2 Response procedure (summary):

1. Detect — via `admin.audit-log.tsx` review, RLS policy denials, `audit_archive_runs` errors, edge-function logs, or user report.
2. Contain — administrator revokes the affected user's role (`user_roles delete` → `is_approved()` false immediately), resets password, and suspends access (`admin-users.functions.ts`).
3. Assess — determine scope from `audit_log` (full before/after JSONB snapshots), `shr_transmission_log`, and `dha_outbound_queue`.
4. Notify — data-breach notification to ODPC within 72 hours and to affected data subjects where required (Kenya Data Protection Act, 2019 s.43); DHA/SHA notification where the incident involves the HIE or claims.
5. Remediate — apply migration fix, re-run verification SQL from `20260811000007_security_hardening.sql`, rotate secrets.
6. Post-incident — record in the facility breach register; review policy.

9.3  GAP (administrative): a signed incident-response plan with named roles, and a breach register, must be maintained by the facility and presented at audit. This policy section is the control; the register is an operational artefact to keep at the facility.

| Section 10 — Accepted Risks & Mitigations

#	Risk	Accepted?	Mitigation / rationale
1	anon EXECUTE warning on SECURITY DEFINER functions required by the Lovable platform	Accepted (platform limitation)	Functions return no patient data anonymously; search_path pinned; EXECUTE revoked from anon wherever functionally possible; reviewed quarterly
2	MFA not yet enforced	Accepted (interim)	30-minute session timeout + strong password policy + RLS; MFA planned on Supabase plan upgrade
3	Server-side password policy not configured	Accepted (interim)	Client-side Zod policy enforced; server policy activated on Supabase transfer
4	On-screen OTP display (Track A) instead of SMS	Accepted (interim)	OTPs hashed (SHA-256), 10-minute expiry, only shown to the receptionist at point of care; SMS (Track B) pending Africa's Talking credentials
5	Permissive CORS on edge functions	To be tightened	Restrict to application origins before certification
6	rooms.\$id.tsx 4 pre-existing ESLint warnings (lines 325, 2269, 4211, 5487)	Accepted	Pre-existing, intentional, documented; do not affect security posture

Review & Approval

Role	Name	Signature	Date
Prepared by	(Technical Lead)		
Reviewed by	(Facility In-Charge)		
Approved by	(Board / Management)		

Review cycle: annually, or upon material change to authentication, authorisation, or data-sharing arrangements.

End of DOC-6.